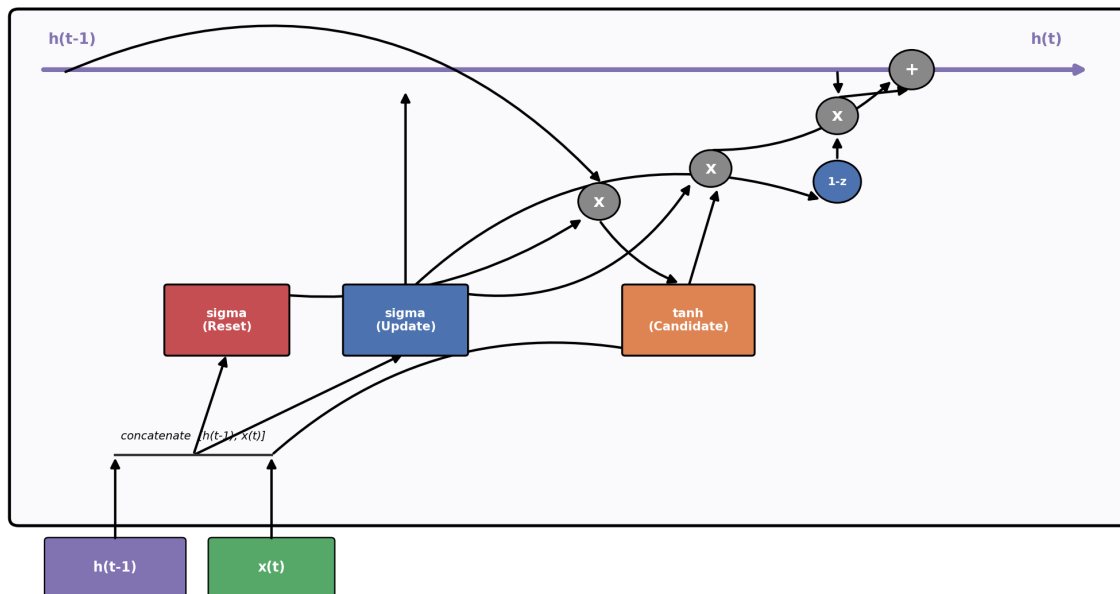


# GRU

## Gated Recurrent Unit

*The Idea, The Input  $x(t)$ , The Hidden State, The Reset Gate & How It Compares to LSTM*

**The GRU Cell - Full Architecture**



Covers: The idea behind GRU • The input  $x(t)$  • What the hidden state is • Full architecture • Calculating the reset gate • The update gate & candidate state • LSTM vs GRU

# 1. The Idea Behind GRU

The **Gated Recurrent Unit (GRU)**, introduced by Cho et al. in 2014, is a streamlined alternative to the LSTM. It was designed to solve the exact same problem - the vanishing gradient / long-term dependency problem in vanilla RNNs - but with a simpler internal structure that trains faster and uses fewer parameters.

The key simplification: instead of keeping **two** separate pieces of memory (the cell state  $C(t)$  and the hidden state  $h(t)$ , as LSTM does), GRU merges them into a **single** hidden state  $h(t)$  that carries all the memory forward.

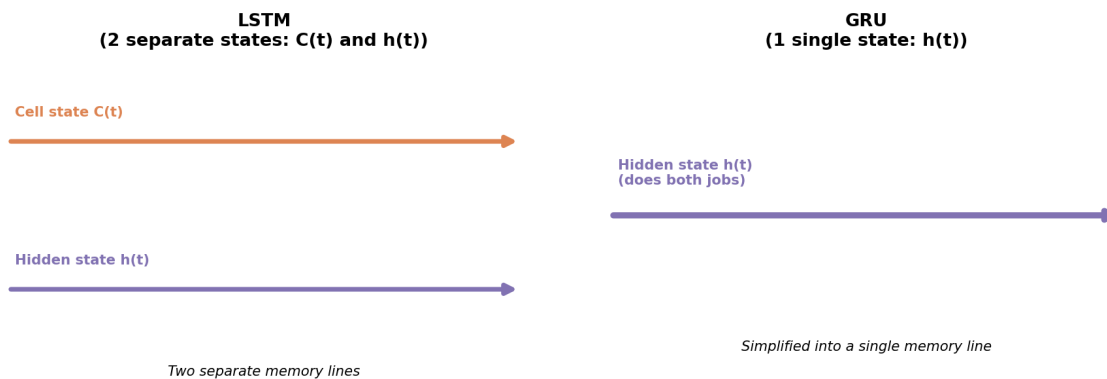


Figure 1: LSTM tracks two separate states; GRU merges everything into one hidden state.

GRU also reduces the number of gates from three (in LSTM) down to **two**: a **reset gate** and an **update gate**. Fewer gates means fewer weight matrices to learn, which is the main reason GRUs are quicker to train than LSTMs while often achieving very similar accuracy.

## 2. The Input $x(t)$ and the Hidden State

### 2.1 The Input $x(t)$

Just like in a vanilla RNN or an LSTM,  $x(t)$  is simply the input vector fed into the network at the current time step  $t$ . Depending on the task, this could be a word embedding, a numeric feature vector, a sensor reading, or any other vectorized representation of one element in a sequence.

### 2.2 What Is the Hidden State?

The **hidden state  $h(t)$**  is GRU's single running memory - a vector that summarizes everything the network has seen in the sequence up to time step  $t$ . At every time step, GRU:

- Looks at the new input  $x(t)$  and the previous hidden state  $h(t-1)$ ,
- Decides (via its gates) how much of the old memory to keep vs. how much new information to blend in,

- Produces an updated hidden state  $h(t)$ , which becomes both the output at this step AND the memory passed to the next step.

Unlike LSTM's separate cell state, GRU's  $h(t)$  has to do double duty - it's simultaneously the long-term memory carrier and the value used to compute the current output.

### 3. Notation Legend

---

#### GRU Notation Legend

|                |                                                                   |
|----------------|-------------------------------------------------------------------|
| $x(t)$         | Input vector at the current time step                             |
| $h(t-1)$       | Hidden state (memory) carried from the previous time step         |
| $h(t)$         | New hidden state - the ONLY state GRU carries forward             |
| $r(t)$         | Reset gate - decides how much past hidden state to forget         |
| $z(t)$         | Update gate - decides how much of $h(t-1)$ vs new content to keep |
| $\tilde{h}(t)$ | Candidate hidden state - proposed new content                     |

*Figure 2: Every symbol used in the GRU cell, defined in one place.*

## 4. The Full GRU Architecture

Here's the complete picture: at every time step,  $x(t)$  and  $h(t-1)$  are combined and fed into the reset gate and update gate. The reset gate influences how the candidate hidden state is computed, and the update gate decides the final blend between old memory and new candidate content.

**The GRU Cell - Full Architecture**

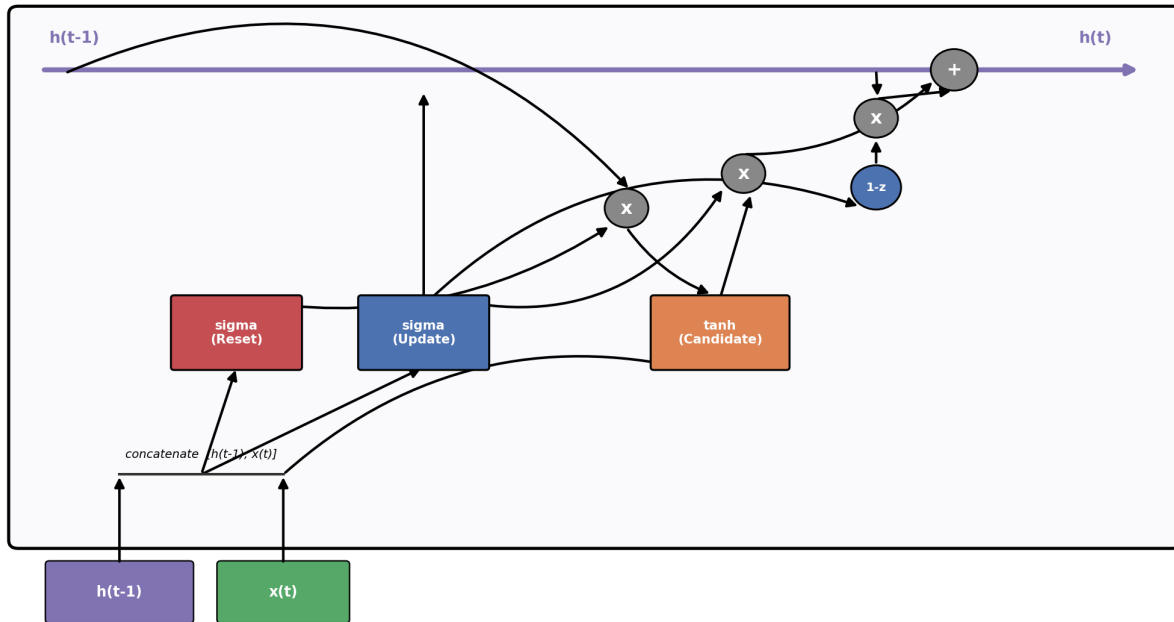


Figure 3: The complete GRU cell - reset gate, update gate, candidate hidden state, and final blending.

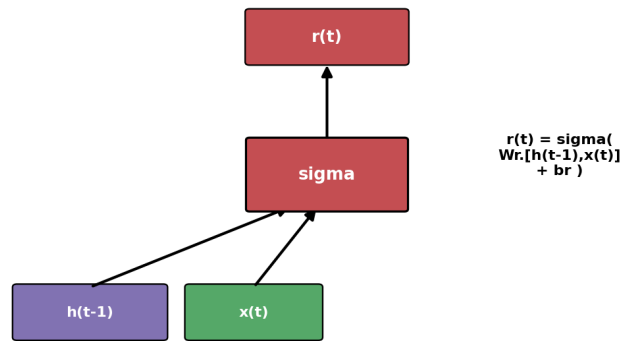
The GRU computation happens in three stages:

- **Stage 1:** Compute the reset gate  $r(t)$  and update gate  $z(t)$  from  $x(t)$  and  $h(t-1)$ .
- **Stage 2:** Use  $r(t)$  to compute a candidate hidden state  $\tilde{h}(t)$  - the 'proposed' new content.
- **Stage 3:** Use  $z(t)$  to blend  $h(t-1)$  and  $\tilde{h}(t)$  together into the final  $h(t)$ .

## 5. Calculating the Reset Gate $r(t)$

The **reset gate** decides how much of the previous hidden state  $h(t-1)$  should be 'forgotten' when computing the new candidate content. It looks at both  $h(t-1)$  and  $x(t)$ , passes their concatenation through a sigmoid layer, producing values between 0 and 1.

### Calculating the Reset Gate $r(t)$



Output between 0 and 1: controls how much of  $h(t-1)$  is used when computing the candidate

Figure 4: The reset gate takes  $h(t-1)$  and  $x(t)$ , and outputs a value between 0 and 1 per unit.

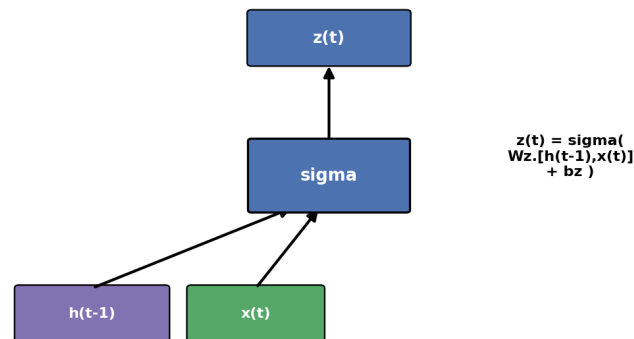
$$r(t) = \text{sigma}(W_r \cdot [h(t-1), x(t)] + b_r)$$

- If  $r(t)$  is close to **0**, the model mostly ignores the previous hidden state - useful when starting a new, unrelated piece of information (e.g. a new sentence or clause).
- If  $r(t)$  is close to **1**, the model keeps most of the previous hidden state when forming the candidate.

## 6. The Update Gate $z(t)$

The **update gate** decides how much of the old hidden state to carry forward versus how much of the new candidate content to blend in. It plays a role similar to the combination of LSTM's forget gate and input gate, but merged into a single gate.

Calculating the Update Gate  $z(t)$



Output between 0 and 1: controls the balance between old memory and new candidate content

Figure 5: The update gate also takes  $h(t-1)$  and  $x(t)$ , producing another value between 0 and 1.

$$z(t) = \text{sigma}(W_z \cdot [h(t-1), x(t)] + b_z)$$

## 7. The Candidate Hidden State $\tilde{h}(t)$

The candidate hidden state represents the new content the network might add to memory. It uses the reset gate to control how much of the past hidden state  $h(t-1)$  influences this candidate, then passes everything through a tanh layer.

### Calculating the Candidate Hidden State $\tilde{h}(t)$

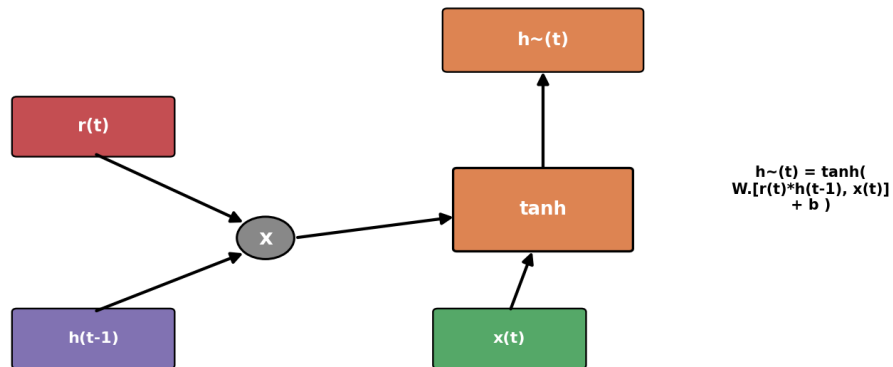


Figure 6: The reset gate first scales down  $h(t-1)$ , then the result is combined with  $x(t)$  and squashed by  $\tanh$ .

$$\tilde{h}(t) = \tanh(W \cdot [r(t) * h(t-1), x(t)] + b)$$

## 8. Final Step - Producing $h(t)$

Finally, the update gate  $z(t)$  linearly blends the old hidden state  $h(t-1)$  and the new candidate  $\tilde{h}(t)$  to produce the final hidden state for this time step:

### Final Step - Combining Old Memory & New Candidate

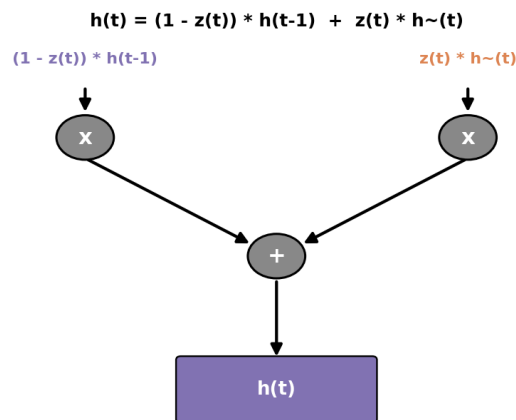


Figure 7:  $z(t)$  acts like a dial - close to 0 keeps the old state, close to 1 adopts the new candidate.

$$h(t) = (1 - z(t)) * h(t-1) + z(t) * \tilde{h}(t)$$

This single equation is what lets GRU (like LSTM) preserve information over long sequences: when  $z(t)$  stays close to 0 for many time steps,  $h(t-1)$  passes through almost unchanged, avoiding the repeated-multiplication decay that plagues vanilla RNNs.

## 9. LSTM vs GRU

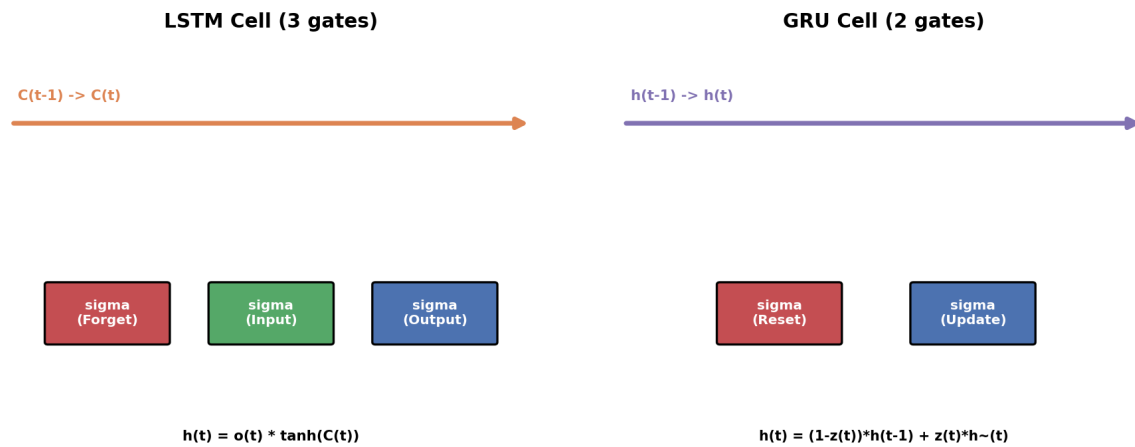


Figure 8: Structural comparison - LSTM's 3 gates and 2 states vs GRU's 2 gates and 1 state.

### LSTM vs GRU - Quick Comparison

| Aspect              | LSTM                                    | GRU                                            |
|---------------------|-----------------------------------------|------------------------------------------------|
| Number of gates     | 3 (forget, input, output)               | 2 (reset, update)                              |
| States carried      | Cell state $C(t)$ + hidden state $h(t)$ | Only hidden state $h(t)$                       |
| Parameters          | More (4 weight sets)                    | Fewer (3 weight sets) -> faster to train       |
| Long-range memory   | Very strong, fine-grained control       | Strong, slightly less fine-grained             |
| Training speed      | Slower                                  | Faster (~25-30% fewer params)                  |
| Typical performance | Excellent, esp. very long sequences     | Comparable on most tasks, great default choice |

Figure 9: Side-by-side comparison of LSTM and GRU across key dimensions.

### When to choose which?

- **Choose GRU** when you want faster training, a smaller model, and your sequences aren't extremely long - it's a great default starting point.
- **Choose LSTM** when you need the most fine-grained control over memory (separate cell state), are working with very long sequences, or have enough compute budget that speed is less of a concern.

- In practice, the performance difference between the two is often small and task-dependent - many practitioners try both and pick whichever performs better via experimentation.

## 10. Summary - All Equations Together

| Component                | Equation                                             | Role                                         |
|--------------------------|------------------------------------------------------|----------------------------------------------|
| Reset gate $r(t)$        | $\text{sigma}(W_r \cdot [h(t-1), x(t)] + b_r)$       | How much of $h(t-1)$ to use in the candidate |
| Update gate $z(t)$       | $\text{sigma}(W_z \cdot [h(t-1), x(t)] + b_z)$       | Balance between old memory and new candidate |
| Candidate $\tilde{h}(t)$ | $\text{tanh}(W \cdot [r(t) \cdot h(t-1), x(t)] + b)$ | Proposed new content                         |
| Hidden state $h(t)$      | $(1 - z(t)) \cdot h(t-1) + z(t) \cdot \tilde{h}(t)$  | Final memory / output for this time step     |

Both LSTM and GRU solve the long-term dependency problem using the same core trick - gating mechanisms that let gradients flow through many time steps largely unchanged. GRU simply achieves this with a leaner design, making it a fast, effective default choice for many sequence modeling tasks.